

Project Title: TempTalk – Real-Time Chat with AWS Deployment

Course Name: Cloud Computing

Submitted in partial fulfillment of the requirements of the
degree

BACHELOR OF ENGINEERING
IN
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

By

Salomi Samuel Raj - 123A8050

Supervisor

Prof. Akshay Anantrao Jadhav



Department of
Artificial Intelligence and Data Science
SIES Graduate School of Technology Nerul,
Navi Mumbai - 400 706

University of Mumbai
(AY 2025-26)

Date of Submission: 07 April 2026

ABSTRACT

TempTalk is a real-time chat application designed for quick and flexible communication in dynamic environments. Many traditional messaging platforms require persistent sessions and complex authentication, which may not be suitable for temporary or instant interactions. This project addresses that gap by providing a lightweight, room-based chat system where users can join and exit conversations effortlessly.

A key feature of TempTalk is its session reset mechanism, where users can instantly return to the join screen by refreshing the page. This provides a layer of privacy and user control, especially when using shared or temporary devices. While the client session resets, chat data is securely maintained at the server and database level to ensure consistency and reliability.

The application leverages cloud technologies for scalability and performance. The backend is built using Node.js and Socket.IO and deployed using AWS Elastic Beanstalk. Chat data is stored in AWS RDS (MySQL), while AWS S3 is used for storing images and file attachments. The expected outcome is a scalable, efficient, and user-friendly chat system that demonstrates real-time communication combined with cloud-based architecture.

1. INTRODUCTION

1.1 Background of the Problem

In today's digital world, communication applications are widely used for collaboration and interaction. However, many existing chat platforms require complex user authentication, permanent accounts, and continuous sessions, which are not always suitable for quick or temporary conversations. In scenarios such as shared devices, classroom discussions, or short-term collaborations, users need a simple and flexible communication system without long-term commitments.

1.2 Importance of Cloud Computing

Cloud computing plays a crucial role in building scalable and efficient applications. It allows developers to deploy applications without managing physical infrastructure. Services like cloud-based servers, databases, and storage provide high availability, reliability, and scalability. In this project, cloud technologies ensure that the chat system can handle multiple users, store data securely, and support real-time communication efficiently.

1.3 Purpose of the Project

The purpose of TempTalk is to develop a lightweight, real-time chat application that enables users to communicate instantly without complex setup. The system focuses on ease of use, privacy, and flexibility by allowing users to join chat rooms quickly and reset their sessions easily. Additionally, the project demonstrates the practical implementation of cloud services for building modern web applications.

2. PROBLEM STATEMENT

Many existing chat applications are designed for long-term usage and require user authentication, persistent sessions, and complex interfaces. This creates challenges for users who need quick, temporary, and flexible communication.

The problem addressed in this project is to design and develop a real-time chat system that:

- Allows users to join and leave conversations instantly
- Supports temporary communication without strict session persistence
- Ensures data consistency and reliability despite client-side resets
- Utilizes cloud technologies for scalability, storage, and performance

TempTalk solves this problem by providing a simple, room-based chat platform with a session reset mechanism, enabling efficient and user-friendly communication in dynamic environments.

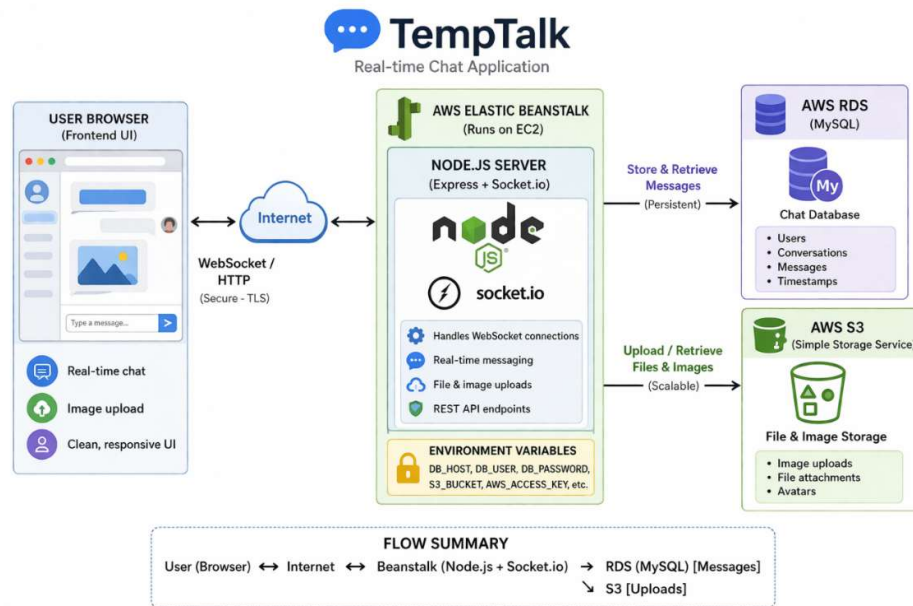
3. OBJECTIVES

The main objectives of the TempTalk project are:

- To design and deploy a cloud-based real-time web application on AWS
- To implement scalable compute infrastructure using Elastic Beanstalk and EC2
- To utilize Amazon RDS as a managed database service for persistent message storage
- To integrate Amazon S3 for secure file and image storage with delete functionality
- To implement per-user message deletion tracked in the cloud database
- To demonstrate real-time communication using Socket.IO over cloud infrastructure
- To configure IAM roles and security groups for secure cloud access control

4. SYSTEM ARCHITECTURE

Architecture Diagram



Explanation of Components

1. Client Browser

The client browser is the user interface of the application. Users can enter a username and room name to join chats, send messages, and upload files. It communicates with the backend using Socket.IO for real-time interaction.

2. Web Server (Node.js on Elastic Beanstalk)

The backend server is built using Node.js and runs on AWS Elastic Beanstalk. It handles:

- User connections
- Room management
- Message broadcasting
- API requests

Elastic Beanstalk automatically manages deployment, scaling, and server infrastructure.

3. EC2 Instance (Underlying Server)

Elastic Beanstalk internally uses EC2 instances to run the application. This acts as the actual computing resource where the Node.js server executes.

4. Database (AWS RDS – MySQL)

AWS RDS is used to store chat data such as:

- Messages
- Usernames
- Room details
- Timestamps

It ensures data persistence, consistency, and reliability even when users leave or refresh the application.

5. Storage (AWS S3)

AWS S3 is used to store:

- Image uploads
- File attachments

Instead of storing files in the database, only the file URLs are saved, improving performance and scalability.

6. Security Layer

Security is handled through:

- AWS Security Groups (control access to EC2 and RDS)
- Environment variables for sensitive data (DB credentials, AWS keys)
- Controlled IP access for database connections

5. CLOUD SERVICES USED

Table: Cloud Services and Their Purpose

Service Type	Cloud Service	Purpose
IaaS	AWS EC2	Hosts the Node.js application server, auto provisioned by EBS
PaaS	AWS Elastic Beanstalk	Manages deployment, scaling, and environment configuration
DBaaS	Amazon RDS (MySQL)	Stores chat messages, room data, and per-user deletion records
Storage as a Service	AWS S3	Stores uploaded images and files shared in chat
Security as a Service	AWS IAM	Controls access to S3 using access keys and secret keys
	Security Groups	Restricts RDS access to only EBS EC2 and authorized IPs

6. SYSTEM IMPLEMENTATION

Frontend Development

The frontend of TempTalk is developed using HTML, CSS, and JavaScript. It provides a simple and user-friendly interface where users can enter a username and room name to join a chat. The interface supports real-time message display and file uploads. Socket.IO is used on the client side to establish real-time communication with the server.

Backend Logic

The backend is built using Node.js and Express. Socket.IO is used to handle real-time communication between users. The server manages:

- User connections and room joining
- Message broadcasting
- File upload handling
- Interaction with database and storage services

The server manages room-based broadcasting, ensuring messages are only sent to users in the same room. Multer is used for handling multipart file uploads before forwarding files to S3.

Database Design

Two tables are used in the temptalk MySQL database. The messages table stores all chat messages with columns for id, room, username, message, and created_at timestamp. The deleted_for table tracks per-user message deletions with columns for id, message_id, and username, with a unique constraint to prevent duplicate entries.

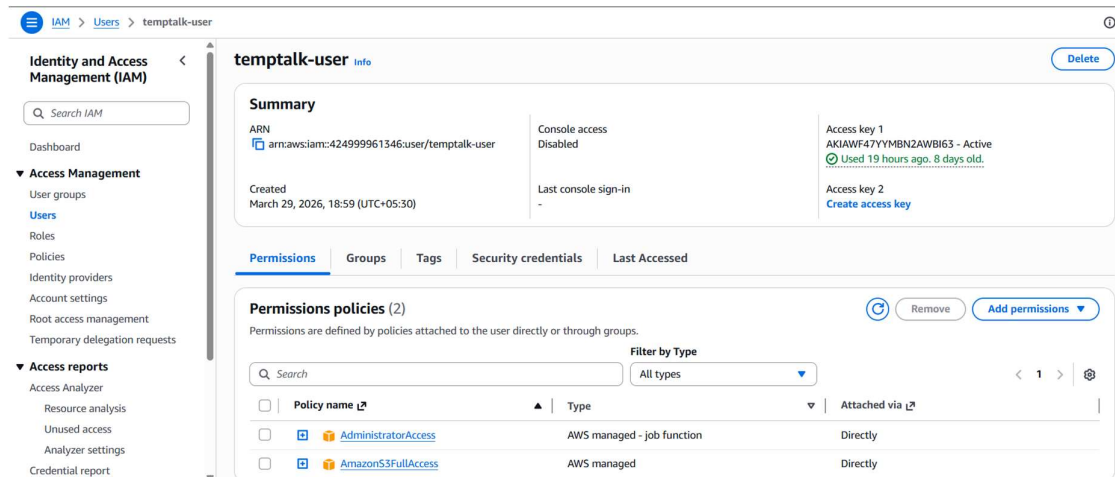
API Integration

The /upload POST endpoint accepts file uploads, streams them to S3 using the AWS SDK v3 PutObjectCommand, and returns the public S3 URL. Socket.IO events handle joining rooms, sending messages, typing indicators, online user tracking, and message deletion.

7. DEPLOYMENT PROCESS

1. AWS Account Setup:

- Created an AWS account and configured IAM user with AdministratorAccess and AmazonS3FullAccess policies.
- Generated access keys for programmatic S3 access from the application.



2. RDS Setup:

- Created a MySQL 8.4 RDS instance named temptalk-db in the eu-north-1 region.
- Created the temptalk database and the messages and deleted_for tables.
- Configured a security group temptalk-sg to restrict access.

Aurora and RDS > **Databases** > temptalk-db

temptalk-db Modify Actions

Summary

DB identifier temptalk-db	Status Starting	Role Instance	Engine MySQL Community	Recommendations
CPU -	Class db.t4g.micro	Current activity	Region & AZ eu-north-1a	

Connectivity & security | Monitoring | Logs & events | Configuration | Zero-ETL integrations | Maintenance & backups | Data

Connect using | Info

☒ **Code snippets**
Use when connecting through SDK, APIs, or third-party tools including agents.

☐ **CloudShell**
Use for a quick access to AWS CLI that launches directly from the AWS Management Console.

☐ **Endpoints**
Use when connecting through any IDE interface.

Internet access gateway
☐ Disabled

IAM Authentication
☐ Disabled

Programming language
MySQL (macOS)

Endpoint type
Instance endpoint

Security group rules (3) Filter by Security group rules < 1 > Settings

Security group	Type	Rule
temptalk-sg (sg-0f193bbe365bb4b04)	CIDR/IP - Inbound	183.87.179.101/32
temptalk-sg (sg-0f193bbe365bb4b04)	EC2 Security Group - Inbound	sg-0f45bc79154f4cab6
temptalk-sg (sg-0f193bbe365bb4b04)	CIDR/IP - Outbound	0.0.0.0/0

3. S3 Setup:

- Created an S3 bucket named temptalk-files-123 in eu-north-1 for storing uploaded images.
- Configured bucket permissions to allow public read access for serving images in chat.

Amazon S3 > **Buckets**

Buckets General purpose buckets All AWS Regions Directory buckets

General purpose buckets (4) Info Copy ARN Empty Delete Create bucket

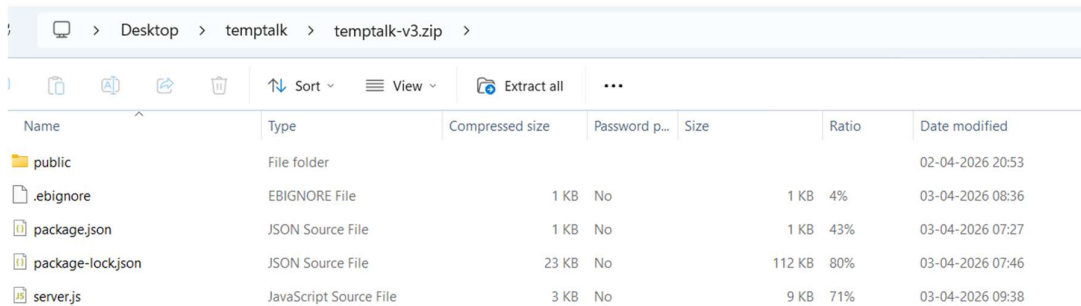
Buckets are containers for data stored in S3.

< 1 > Settings

Name	AWS Region	Creation date
elasticbeanstalk-eu-north-1-424999961346	Europe (Stockholm) eu-north-1	April 2, 2026, 11:02:19 (UTC+05:30)
iot-dashboard-salomi	Europe (Stockholm) eu-north-1	March 31, 2026, 12:30:17 (UTC+05:30)
iot-dashboard-salomi-ireland	Europe (Ireland) eu-west-1	April 3, 2026, 14:47:37 (UTC+05:30)
temptalk-files-123	Europe (Stockholm) eu-north-1	March 29, 2026, 18:54:39 (UTC+05:30)

4. Application Packaging:

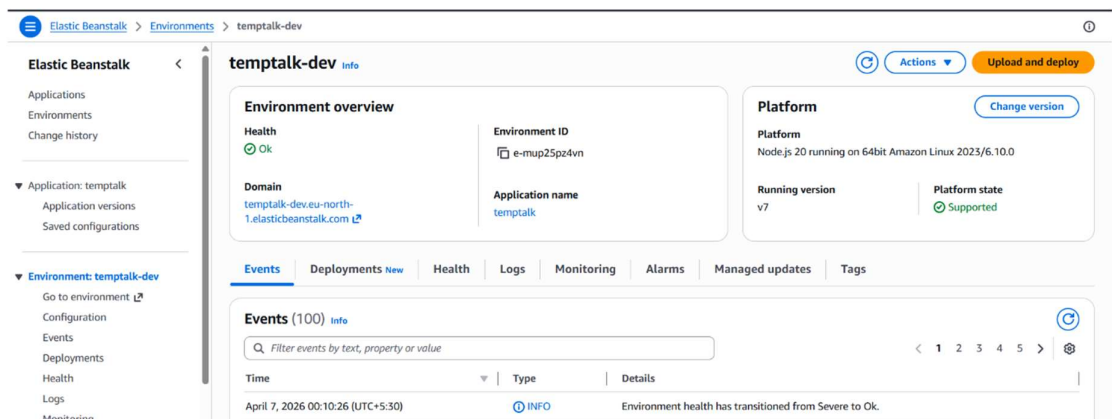
- Prepared the deployment package by zipping server.js, package.json, package-lock.json, and the public/ folder.
- A .ebignore file was used to exclude node_modules, .env, and uploads/ from the package.



Name	Type	Compressed size	Password p...	Size	Ratio	Date modified
public	File folder					02-04-2026 20:53
.ebignore	EBIGNORE File	1 KB	No	1 KB	4%	03-04-2026 08:36
package.json	JSON Source File	1 KB	No	1 KB	43%	03-04-2026 07:27
package-lock.json	JSON Source File	23 KB	No	112 KB	80%	03-04-2026 07:46
server.js	JavaScript Source File	3 KB	No	9 KB	71%	03-04-2026 09:38

5. Elastic Beanstalk Deployment:

- Created an Elastic Beanstalk application named temptalk with environment temptalk-dev using the Node.js 20 platform on 64-bit Amazon Linux 2023.
- Uploaded the zip package via the Upload and Deploy option.



The screenshot shows the AWS Elastic Beanstalk console for the 'temptalk-dev' environment. The left sidebar shows the navigation menu with 'Elastic Beanstalk' selected. The main content area shows the 'Environment overview' for 'temptalk-dev'. The 'Health' status is 'Ok'. The 'Environment ID' is 'e-mup25pz4vn'. The 'Application name' is 'temptalk'. The 'Platform' is 'Node.js 20 running on 64bit Amazon Linux 2023/6.10.0'. The 'Platform state' is 'Supported'. The 'Events' tab is selected, showing a list of events with a filter bar and a table of events.

Time	Type	Details
April 7, 2026 00:10:26 (UTC+5:30)	INFO	Environment health has transitioned from Severe to Ok.

EBS automatically:

- Created an S3 bucket for environment data storage
- Created a security group sg-012b992fda7f96a9d
- Created a target group and Application Load Balancer
- Created an Auto Scaling group with scaling policies
- Created CloudWatch alarms for monitoring (High/Low)
- Provisioned an EC2 t3.micro instance and deployed the application code

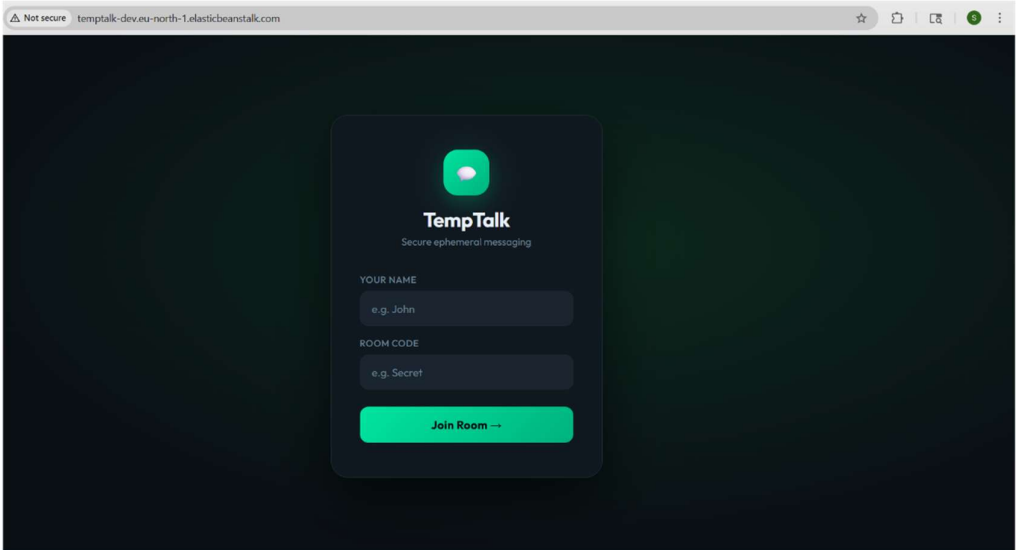
```

would you like to enable Spot Fleet requests for this environment? (y/N):
2.0: Platforms require a service role. We will attempt to create one for you. You can specify your own role using the --service-role option.
Type 'y' to accept the policy, or just press ENTER to continue.
Creating application version archive... app-260402.110928799390
Uploading tempalk/app-260402.110928799390.zip to S3. This may take a while.
Upload Complete.
Environment details for: tempalk-dev
  Application name: tempalk
  Region: eu-north-1
  Deployed Version: app-260402.110928799390
  Environment ID: e-mup25pz4vn
  Platform: arn:aws:elasticbeanstalk:eu-north-1::platform/Node.js 20 running on 64bit Amazon Linux 2023/6.10.0
  Tier: WebServer-Standard-1.0
  CHAME: tempalk-dev, eu-north-1.elasticbeanstalk.com
  Updated: 2026-04-02 05:41:32.985000+00:00

Printing Status:
2026-04-02 05:41:31 INFO createEnvironment is starting.
2026-04-02 05:41:31 INFO Using elasticbeanstalk-eu-north-1-424999961346 as Amazon S3 storage bucket for environment data.
2026-04-02 05:42:00 INFO Created security group named: sg-012b92f647f96a93
2026-04-02 05:42:16 INFO Created target group named: arn:aws:elasticloadbalancing:eu-north-1:424999961346::targetgroup/awseb-AWSEB-7GULPKU
2026-04-02 05:42:16 INFO Created security group named: awseb-e-mup25pz4vn-stack-AWSEBSecurityGroup-TfG6eASysdm
2026-04-02 05:42:47 INFO Created Auto Scaling group named: awseb-e-mup25pz4vn-stack-AWSEBAutoScalingGroup-gdRTxyQDPT0S
2026-04-02 05:42:47 INFO Waiting for EC2 instances to launch. This may take a few minutes.
2026-04-02 05:43:02 INFO Created Auto Scaling group policy named: arn:aws:autoscaling:eu-north-1:424999961346::scalingPolicy:7cf091ce-9e87
2026-04-02 05:43:02 INFO Created Auto Scaling group policy named: arn:aws:autoscaling:eu-north-1:424999961346::scalingPolicy:4d0mnplicy-188c1b-6786
2026-04-02 05:43:02 INFO Created Auto Scaling group policy named: arn:aws:autoscaling:eu-north-1:424999961346::scalingPolicy:1988c1b1-b786
2026-04-02 05:43:02 INFO Created Auto Scaling group policy named: arn:aws:autoscaling:eu-north-1:424999961346::scalingPolicy:c35RPMVVE
2026-04-02 05:43:02 INFO Created CloudWatch alarm named: awseb-e-mup25pz4vn-stack-AWSEBCloudWatchAlarmLow-LDHFajbvFkAn
2026-04-02 05:43:02 INFO Created CloudWatch alarm named: awseb-e-mup25pz4vn-stack-AWSEBCloudwatchAlarmHigh-MTTJX7sQ0MSB
2026-04-02 05:44:34 INFO Created load balancer named: arn:aws:elasticloadbalancing:eu-north-1:424999961346::loadbalancer/app/awseb-AWSEB-
2026-04-02 05:44:35 INFO Created Load Balancer listener named: arn:aws:elasticloadbalancing:eu-north-1:424999961346::listener/app/awseb-A
4056c507c80539
2026-04-02 05:44:46 INFO Instance deployment completed successfully.
2026-04-02 05:45:17 INFO Application available at tempalk-dev, eu-north-1.elasticbeanstalk.com.
2026-04-02 05:45:18 INFO Successfully launched environment: tempalk-dev

```

The application became available at temptalk-dev.eu-north-1.elasticbeanstalk.com after successful deployment.



The EC2 instance can be accessed via EC2 Instance Connect where all project files are visible at `/var/app/current/` including `server.js`, `package.json`, and the `public/` folder.

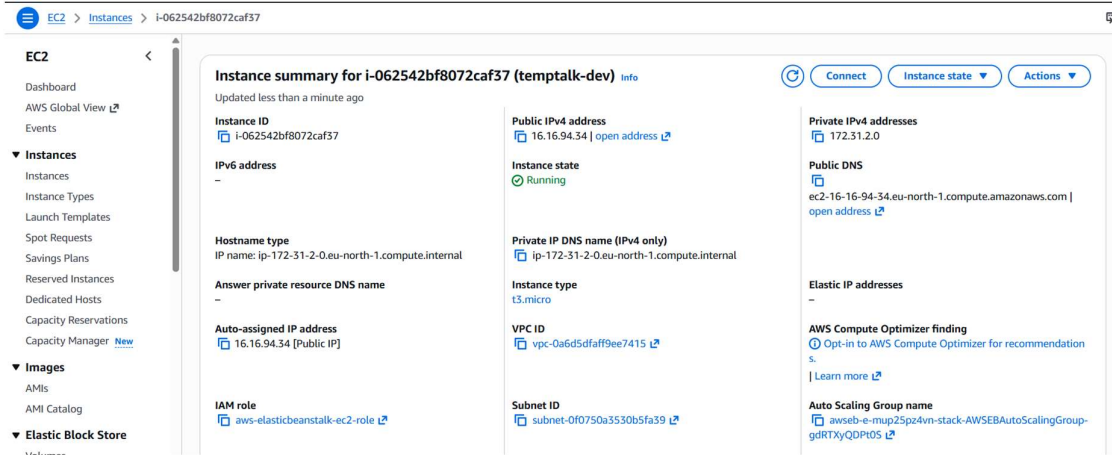
```

Amazon Linux 2023 AMI

This EC2 instance is managed by AWS Elastic Beanstalk. Changes made via SSH
WILL BE LOST if the instance is replaced by auto-scaling. For more information
on customizing your Elastic Beanstalk environment, see our documentation here:
http://docs.aws.amazon.com/elasticbeanstalk/latest/dg/customize-containers-ec2.html

#_
~\#### Amazon Linux 2023
~~\#####\
~~\###|
~~\#/ https://aws.amazon.com/linux/amazon-linux-2023
~~V~'!->
~~~
~~~.
~~~\_/m/'
Last login: Mon Apr 6 15:33:58 2026 from 13.48.4.203
[root@ip-172-31-2-0 ~]# ls /var/app/current/
node_modules package.json package-lock.json Procfile server.js uploads
[root@ip-172-31-2-0 ~]#

```



8. SECURITY IMPLEMENTATION

- **Authentication:** The application uses room codes as a basic access control mechanism — only users who know the room code can join and view messages. IAM access keys are used to authenticate S3 operations from the server.
- **Access Control:** RDS security groups are configured to allow inbound MySQL connections only from the EBS EC2 security group and authorized IP addresses. This ensures the database is not accessible from the public internet.
- **HTTPS:** S3 file URLs are served over HTTPS, ensuring encrypted file transmission. The application can be further secured by adding an SSL certificate via AWS Certificate Manager and a Load Balancer.
- **Data Protection:** Sensitive credentials including database passwords and AWS access keys are stored as Elastic Beanstalk environment properties and never included in the source code or deployment package. The .env file is excluded from deployment using .ebignore.
- **Security Incident & Resolution:** During development, the RDS instance was compromised by an automated bot that exploited publicly accessible port 3306 with a weak password. The incident was resolved by changing the master password to a strong credential, disabling public accessibility, and restricting security group inbound rules to specific IP addresses only.

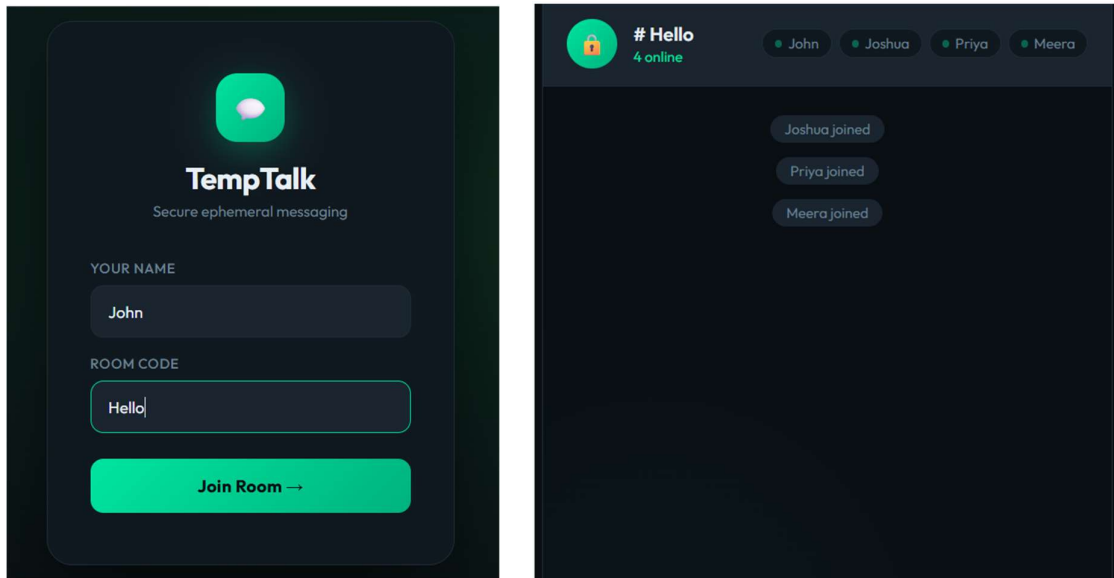
9. RESULTS AND TESTING

Application Results

Test 1 — Real-time Messaging:

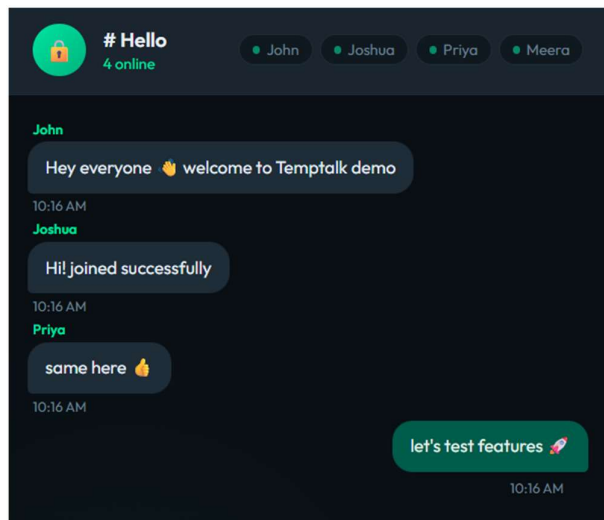
Multiple users joined the same room from different browsers.

Messages sent by one user appeared instantly on the other user's screen without page refresh, confirming real-time Socket.IO communication.



Test 2 — Message Persistence:

After sending messages and refreshing the browser, all previous messages loaded correctly from RDS, confirming persistent cloud database storage.



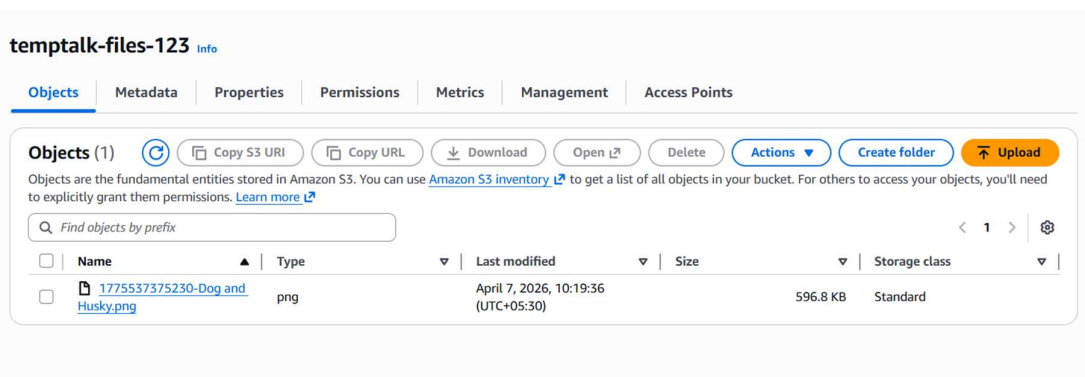
Test 3 — Room Isolation:

Two separate pairs of users joined different rooms simultaneously. Messages from one room did not appear in the other, confirming correct room-based isolation.

Test 4 — Image Upload to S3:

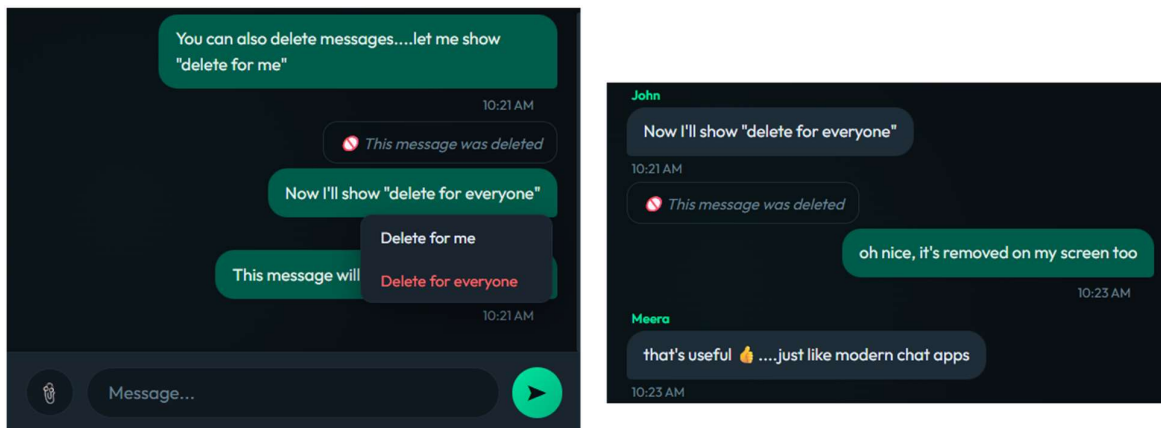
A user uploaded an image in chat.

The image appeared in the chat and was confirmed present in the S3 bucket via the AWS console.

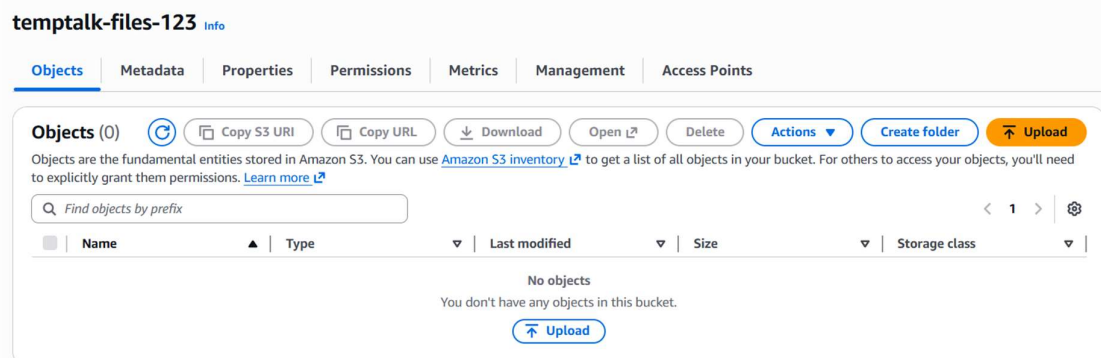
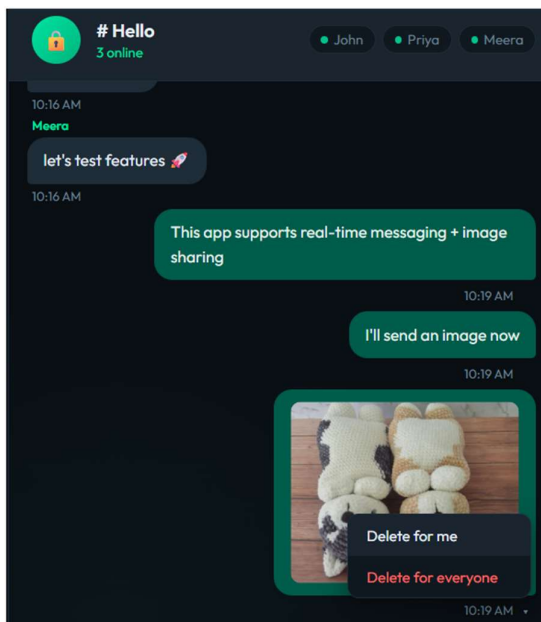


Test 5 — Delete for Everyone:

A user deleted a message for everyone.



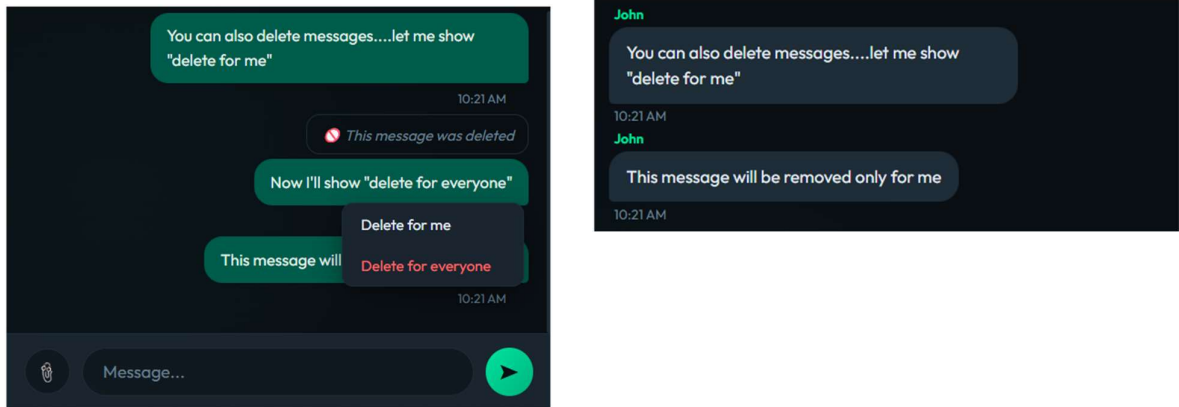
The message was removed from both users' screens, deleted from the RDS messages table, and the associated image was deleted from S3.



Test 6 — Delete for Me:

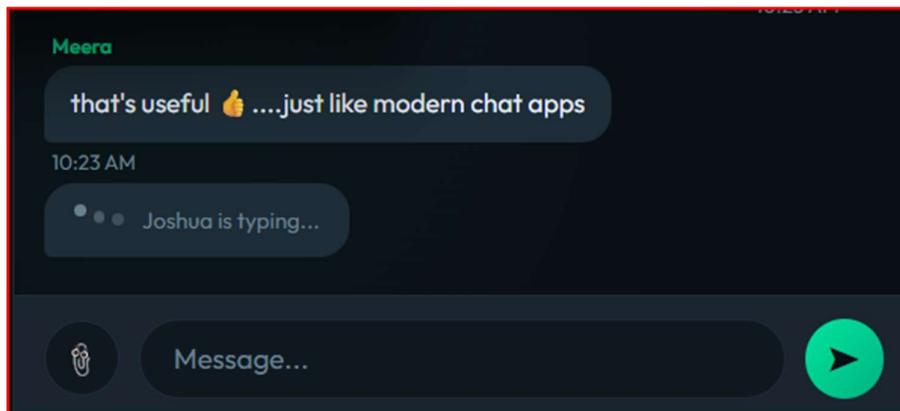
A user deleted a message for themselves only.

After refreshing, the message remained hidden for that user but was still visible to the other user, confirming correct per-user deletion tracking via the `deleted_for` table.



Test 7 — Typing Indicator:

When a user typed a message, the other user saw a typing animation, confirming real-time typing indicator functionality.



Test 8 — Checking Database

```
C:\Users\seren>C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe -h temptalk-db.cdo0g680eh00.eu-north-1.rds.amazonaws.com -u admin -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 37
Server version: 8.4.7 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE temptalk;
Database changed
mysql> SELECT * FROM messages;
```

	id	room	username	message	created_at
33	Hello	John		Hey everyone ? welcome to Temptalk demo	2026-04-07 04:46:08
34	Hello	Joshua		Hi! joined successfully	2026-04-07 04:46:19
35	Hello	Priya		same here ?	2026-04-07 04:46:26
36	Hello	Meera		let's test features ?	2026-04-07 04:46:34
37	Hello	John		This app supports real-time messaging + image sharing	2026-04-07 04:49:16
38	Hello	John		I'll send an image now	2026-04-07 04:49:24
40	Hello	Joshua		got the image instantly ?	2026-04-07 04:49:46
41	Hello	Priya		nice, quality is good too	2026-04-07 04:49:56
42	Hello	John		You can also delete messages....let me show "delete for me"	2026-04-07 04:51:20
43	Hello	John		This message will be removed only for me	2026-04-07 04:51:29
44	Hello	John		Now I'll show "delete for everyone"	2026-04-07 04:51:40
46	Hello	Priya		oh nice, it's removed on my screen too	2026-04-07 04:53:00
47	Hello	Meera		that's useful ?....just like modern chat apps	2026-04-07 04:53:36

```
13 rows in set (0.15 sec)

mysql> SELECT * FROM deleted_for;
```

	id	message_id	username
1		43	John

10. CHALLENGES FACED

- 1. Local vs Cloud Database Confusion** Initially, the application was connecting to a local MySQL instance instead of the AWS RDS instance. This caused significant debugging time as the local database showed different data than the cloud database. The issue was resolved by verifying connection strings and using the correct RDS endpoint in the environment variables.
- 2. S3 File Deletion with Special Characters** When deleting uploaded files from S3, filenames containing spaces or special characters (e.g., Salomi Sign.jpg) were URL-encoded as %20 in the message URL. The S3 delete operation was sending the encoded key instead of the decoded filename, causing deletion to fail silently. This was resolved by applying `decodeURIComponent()` to the extracted S3 key before sending the delete command.
- 3. RDS Security Group Misconfiguration** The RDS instance was initially set to publicly accessible with port 3306 open to 0.0.0.0/0, making it vulnerable. A automated bot attacked the database, wiped all tables, and left a ransom note demanding Bitcoin payment. This was resolved by changing the master password, restricting inbound rules to only allow the EBS security group and specific IP addresses.

```
| Dear Sir/Madam,  
  
We hope this message finds you well.  
  
We would like to inform you that we have backed up your databases/tables. We keep them for 30 days and then will be publicly disclosed and permanently delete them from our servers.  
  
We offer two options:  
  
Option 1.: recovery service + leak protection  
  
If you want to recover your corrupted or incomplete databases/tables and you want to prevent it from being leaked, simply transfer 0.018 BTC to this address:  
  
18U1L28KfJ6WUScc6jpnLfEcXyPtWQyNM  
  
This address is assigned to your database credentials (host + user: 51.21.58.220 + admin). We will know when you have paid.  
  
After payment confirmation, our program will restore the entire databases/tables automatically, so please do not change your database login details and make sure the database is still  
All your databases and tables will be restored. (If the restore fails the current field will be modified and provide links for you to download your datas). The saved files will be  
  
Option 2.: leak protection only  
  
If you do not need to restore your databases but you want to prevent it from being leaked, send 0.007 BTC to this address:  
  
18U1L28KfJ6WUScc6jpnLfEcXyPtWQyNM  
  
In this case, your datas will not be made public and will be deleted from our servers without any recovery service.  
  
-----  
Please take note of the following:  
  
After 30 days, we cannot guarantee the restoration of your data.  
  
The only way to recover your data (and prevent it from being leaked) is by making the payment.  
  
Data leaks can have serious consequences. Rest assured, your data is protected.  
  
Once your payment is completed, all your data will be deleted from our servers. Currently, government agencies, competitors, contractors, and local media remain unaware of the incidents  
Entities will not be contacted about this matter, ensuring your privacy and the confidentiality of the situation are maintained.  
  
Pay now, and we guarantee that your data will not be sold on dark web resources or used to attack your company, employees, or counterparties in the future. The full database dump was  
ted from our servers.  
  
If you do not send the requested amount within 30 days from the date of the incident, we will consider the transaction incomplete. Your data will then be sent to any interested party.  
After payment confirmation, our system will automatically restore the entire databases/tables, so please do not change your database login details and make sure the database is still  
  
-----  
The only accepted payment method is Bitcoin. To the wallet specified above.  
  
Be advised: PayPal, WeTransfer, Alipay, credit cards, and other methods will not be accepted.  
  
If you do not have Bitcoin, you can purchase it using a credit card from the following websites:  
  
Coinbase: https://www.coinbase.com/
```

4. Security Group Blocking EBS-RDS Communication After restricting RDS access to only the local IP for security, the EBS EC2 instance lost access to RDS causing ETIMEDOUT errors in production. This was resolved by adding the EBS security group (awseb-e-mup25pz4vn-stack-AWSEBSecurityGroup) as an inbound rule in the RDS security group, allowing internal AWS communication without exposing the database publicly.

11.CONCLUSION

TempTalk successfully demonstrates the design and deployment of a cloud-native real-time web application using AWS services. The project provided hands-on experience with core cloud computing concepts including Infrastructure as a Service, Platform as a Service, Database as a Service, and Storage as a Service. The application is fully functional with real-time messaging, image sharing, message deletion with S3 cleanup, and per-user deletion persistence — all running on AWS cloud infrastructure. The project also provided valuable lessons in cloud security through a real-world security incident encountered during development.

12.FUTURE ENHANCEMENTS

- Implement HTTPS using AWS Certificate Manager and an Application Load Balancer for encrypted communication
- Add user authentication with password-based login stored in RDS to replace room-code-only access
- Integrate Amazon CloudFront as a CDN in front of S3 for faster image delivery globally
- Implement Auto Scaling in Elastic Beanstalk to handle increased user load automatically
- Add message read receipts and push notifications using AWS SNS
- Implement end-to-end encryption for messages before storing in RDS
- Add support for voice and video calls using WebRTC

13.REFERENCES

- AWS Elastic Beanstalk Documentation: <https://docs.aws.amazon.com/elasticbeanstalk/>
- Amazon RDS Documentation: <https://docs.aws.amazon.com/rds/>
- Amazon S3 Documentation: <https://docs.aws.amazon.com/s3/>
- AWS SDK for JavaScript v3: <https://docs.aws.amazon.com/sdk-for-javascript/>
- Socket.IO Documentation: <https://socket.io/docs/>
- Node.js Documentation: <https://nodejs.org/en/docs/>
- MySQL Documentation: <https://dev.mysql.com/doc/>
- Express.js Documentation: <https://expressjs.com/>

